# TP 2

## 1. Estudie en XINU para que sirven las siguientes system calls (puede revisar el código fuente de XINU, o el libro XINU): suspend(pid); resume(pid); getprio(pid); chprio(pid, newprio); getpid(); sleepms(ms);

- suspend() Suspende un proceso, poniendolo en "hibernación" (PR_SUSP).
  Solo será suspendido un proceso que esta listo o en ejecución.
- getprio(pid) Permite que el llamador obtenga la prioridad de planificación de un proceso arbitrario con su id =pid.
- chprio(pid,newprio) PCambia la prioridad de planificación de un proceso.
- getpid() Retorna la ID del proceso en ejecución.
- resume() Quita un proceso del estado SUSP, poniendolo en listo.
- sleepms(ms) Retarda el proceso la cantidad de ms especificada, insertando el proceso en la lista delta de procesos durmiendo. Su estado es cambiando a PR_SLEEP. Si ms = 0, no lo retarda. La función sleep() invoca a la función sleepms, multiplicando su argumento por por mil.
- create() Crea un nuevo proceso para que empiece a ejecutarse en x86.

## 1.a ¿Se puede hacer un suspend(pid) del proceso pid estando el proceso pid en estado “sleeping”?

No, la misma función suspend indica que solo se suspenderá si estpa en ejecución o listo. Si se realiza esto, una vez que el proceso termine su tiempo de delay y esté en estado listo, en ese instante se suspenderá. Probar en xinu.

## b. Repase el TP anterior: ¿cuánto vale el quantum en XINU? ¿Qué problema o comportamiento puede haber si usted cambia el quantum a 100 ms?

El quantum en Xinu vale 2 ms. El comportamiento que habrá si el quantum se cambia a 100 ms, será que los procesos tendrán más tiempo de ejecución en la CPU, esto hará que el SO realice menos cambios cambios de contexto en un tiempo determinado, pero a su vez causará que exista menos concurrencia entre los procesos de igual prioridad, aumentará el tiempo de **turnaround promedio** entre los procesos cortos, y podrá causar inanición de algún proceso de baja prioridad. Xinu es RR con prioridades apropiativo.

## c. Investigue cómo se llaman y cómo funcionan los algoritmos de planificación de procesos del sistema operativo

# Windows y Linux. Redacte una respuesta-aprendizaje. Esto implica: no buscar en google/chatgpt y sólo copiar y pegar. Intente dar una explicación general. Ayuda: Abraham Silberschatz.

## Enfoques de planificación de sistemas multiprocesadores.

***Multiprocesamiento asimétrico***: delega todas las decisiones de programación, procesamiento de E/S y otras actividades del sistema en un ***único procesador*** (el **servidor maestro**), mientras que los demás procesadores solo ejecutan código de usuario.
Este multiprocesamiento asimétrico es simple porque un sólo núcleo accede a las estructuras de datos del sistema, reduciendo la necesidad de compartir datos, sin embargo el servidor maestro se convierte en un cuello de botella.

***Multiprocesamiento asimétrico***: El enfoque estándar para soportar multiprocesadores es el **procesamiento simétrico (SMP, symmetric multiprocessing)**, donde cada procesador se autoplanifica. La planificación funciona de la siguiente manera: el planificador de cada procesador examina la **cola de listos (ready queue)** y selecciona un hilo para ejecutar.
Esto ofrece dos posibles estrategias para organizar los hilos elegibles para planificación

1. **Cola única compartida**: Todos los procesadores acceden a una misma cola de hilos listos.
2. **Colas múltiples independientes**: Cada procesador tiene su propia cola de hilos asignados

## Planificación de Linux - *"Complete Fair Scheduling"*

CFS es el algoritmo default de Linux. La planificación en el sistema de linux está basada en "clases de planificación", dónde a cada clase se le asigna una prioridad específica, así el Kernel puede adaptarse según las necesidades del sistema y sus procesos (El criterio de planificación de un servidor de Linux, por ejemplo, debe ser diferente a el criterio de planificación de aquellos con un dispositivo móbil corriendo Linux)
Para decidir cuál tarea ejecutar, el planificador selecciona la tarea de mayor prioridad perteneciente a la clase de planificación de mayor prioridad.
El Kernel Standard de Linux implementa dos clases de planificación:

1. Una clase de planificación por defecto que utiliza el algoritmo CFS.
2. Una clase de planificación de tiempo real.

## *CFS Algorithm.*

Este planificador CFS asigna una proporción del tiempo de procesamiento de la CPU a cada tarea, esta proporción se calcula basándose en el "nice value"de cada tarea, los valores del nice value van desde -20 a 19, dónde un valor número menor de nice value

indica mayor prioridad. El nice value por defecto es 0.
Las tareas a las que se les asigna un nice value menor, reciben una mayor proporción de tiempo de procesamiento de la CPU.
Este algoritmo utiliza una "latencia objetivo", el cuál es un tiempo dónde cada tarea en estado listo debería ejecutarse al menos una vez. Esta puede modificarse según la cantidad de procesos activos en el sistema.

Cada tarea es almacenada en un árbol binario de búsqueda "negro-rojo", dónde cada clave está basada en el valor del tiempo de ejecución virtual. Las tareas que tienen menor tiempo de ejecución virtual se almacenan a la izquierda, mientras las que tienen mayor tiempo de ejecución virtual se alamcenan a la derecha. El nodo más a la izquierda tiene el valor clave más pequeño, por lo que tiene mayor prioridad.
Cuándo una tarea está en estado "listo" es añadida al árbol. Sí una tarea no es ejecutable (Por ejemplo, esta bloqueada por una operación I/O) es removida.

Dado que el **árbol rojo-negro** está balanceado, encontrar el nodo más izquierdo requiere operaciones de orden **O(log N)** (donde *N* es el número de nodos). Sin embargo, por eficiencia, el planificador de Linux **almacena en caché** este valor en la variable `rb_leftmost`, por lo que determinar la siguiente tarea a ejecutar solo requiere recuperar dicho valor

## *Planificación de Windows*

Windows planifica hilos utilizando un algoritmo de de planificación ***apropiativo***,***basado en prioridades***. El planificador de Windows se asegura que el hilo de mayor prioridad siempre se ejecute. La porción del Kernel que se encarga de esto se llama "despachador".
Un hilo seleccionado para ejecutarse por el despachador, se ejecutará hasta que sea interrumpido por un hilo de mayor prioridad, hasta que termine, hasta que termine su quantum, o hasta que llame una system call que lo bloquee( como una I/O).
El despachador utiliza un esquema de prioridades de 32 niveles, existen dos clases de prioridades:

- Clase de variables. Contiene hilos de prioridad 1-15.
- Clase de tiempo real. Contiene hilos con prioridad 16-31
- En la prioridad 0 se encuentra en ejecución un hilo usado para el manejo de memoria.

El despachador utiliza una cola para cada prioridad y recorre el conjunto de colas desde la más alta hasta la más baja hasta que encuentra un hilo que está listo para ejecutarse.
Si no hay ningún hilo listo para ejecutarse, el despachador ejecuta un "idle thread"

> **Note**
>
> API "Interfaz de Programación de Aplicaciones". Es un conjunto de reglas y protocolos que permiten que las aplicaciones se comuniquen entre sí.

La API de windows se divide en seis clases de prioridades donde un proceso puede pertenecer:
•IDLE PRIORITY CLASS
• BELOW NORMAL PRIORITY CLASS
• NORMAL PRIORITY CLASS
• ABOVE NORMAL PRIORITY CLASS
• HIGH PRIORITY CLASS
• REALTIME PRIORITY CLASS

Los procesos son miembros de la "NORMAL_PRIORITY_CLASS". UN proceso pertenece a esta clase a menos que el padre fuera miembro de la prioridad IDLE.
Las prioridades de todas las clases exceptuando la REALTIME PRIORITY CLASS pueden cambiar con la función SetPriorityCLass().
Un proceso puede tener una prioridad de clase, y una prioridad relativa (que los tipos son las clases mencionadas antes).

La prioridad de un hilo está basada en la prioridad de clase y la prioridad relativa.

| | real-time | high | above normal | normal | below normal | idle priority |
|---|---|---|---|---|---|---|
| time-critical | 31 | 15 | 15 | 15 | 15 | 15 |
| highest | 26 | 15 | 12 | 10 | 8 | 6 |
| above normal | 25 | 14 | 11 | 9 | 7 | 5 |
| normal | 24 | 13 | 10 | 8 | 6 | 4 |
| below normal | 23 | 12 | 9 | 7 | 5 | 3 |
| lowest | 22 | 11 | 8 | 6 | 4 | 2 |
| idle | 16 | 1 | 1 | 1 | 1 | 1 |

**Cuando el tiempo asignado (quantum) de un hilo se agota, este es interrumpido.**

- Si el hilo pertenece a la **clase de prioridad variable**, su prioridad se reduce, pero **nunca por debajo de su prioridad base**. Esto ayuda a limitar el consumo de CPU por parte de hilos intensivos en procesamiento (*compute-bound*).
- El valor base es el valor NORMAL de la prioridad relativa para esa clase.
  REALTIME PRIORITY CLASS —24
  HIGH PRIORITY CLASS —13
  ABOVE NORMAL PRIORITY CLASS —10
  NORMAL PRIORITY CLASS —8
- Cuando un hilo de prioridad variable **sale de una operación de espera (*wait*)**, el planificador (*dispatcher*) **aumenta su prioridad**. La magnitud del aumento depende del tipo de espera:

- Un hilo que esperaba **entrada de teclado** recibe un **aumento grande**.
- Un hilo que esperaba una **operación de disco** recibe uno **moderado**

Esta estrategia tiende a dar buenas respuestas a los hilos interactivos que estan usando el mouse y windows. También permite que los hilos que limitados por E/S mantengan el dispositivo E/S ocupado mientras permiten que los hilos limitads a computación que utilicen ciclos de la CPU.

La ventana con la que está interactuando el usuario recibe un aumento de la prioridad para mejorar su tiempo de respuesta. Cuándo un usuario esta ejecutando un programa interactivo, el sistema necesita proveerle buen rendimiento. POr esto, WIndows aplica una regla especial a los procesos de prioridad normal. Windows diferencia entre:

- Proceso en primer plano. Proceso seleccionado en pantalla.
- Proceso en segundo plano. Proceso no seleccionado.

Regla: Cuándo un proceso pasa a primer plano, windows incrementa su QUANTUM multiplicandolo por un factor, generalmente 3. Esto significa que el proceso en primer plano recibe el triple de tiempo de CPU antes de ser interrumpido en comparación con los procesos en segundo plano.

## Ejercicio 2.

```
#include <xinu.h>
#include <stdio.h>

  void funcionUno(void), funcionDos(void);



  void ejercicio_7(){
  int pidHijo, pidHijoD;



  pidHijo = create(funcionUno, 2048,20,"Proceso hijo 1", 0); //Creo mi
proceso hijo
  pidHijoD = create(funcionDos, 2048, 30, "Proceso hijo 2", 0); //Le damos
mas prioridad

  resume(pidHijo);
  resume(pidHijoD);

  }
```

```
  void funcionDos( void){
  int esPrimo = 1;
  int cantidadPrimos = 0;
  printf("El 2 es primo");

  for( int i = 3; (i <= 1000); i++){
    //Por cada numero, vemos si es divisible x 1 y por si mismo
      for(int j = 2 ; j < i; j++){
        if (( i % j )== 0) { //Si es divisible x otro numero aparte de sí
mismo, no es primo.
          esPrimo = 0;
          break;
          }
        }

        if(esPrimo == 1) {
        printf("EL numero %d es primo",i);
        cantidadPrimos++;

        }else{ esPrimo = 1;}




        if(cantidadPrimos == 100){
        printf("EL PROCESO  DOS SE DUERME");
        sleep(5);


    }

  }


  }


    void funcionUno(void){
    int esPrimo = 1;

    for( int i = 1000; (i <= 5000); i++){
    //Por cada numero, vemos si es divisible x 1 y por si mismo
      for(int j = 2 ; j < i; j++){
```

```
        if ((i % j) == 0) {
          esPrimo = 0;
          break;
          }
        }

        if(esPrimo == 1) {
        printf("EL numero %d es primo",i);

        }else{
        esPrimo = 1;}

      }
    }
```

## Ejercicio 3.

| Proceso | Prioridad | Ráfaga | Arribo a la cola de listos |
|---|---|---|---|
| P1 | 8 | 15 | 0 |
| P2 | 3 | 20 | 0 |
| P3 | 4 | 20 | 20 |
| P4 | 4 | 20 | 25 |
| P5 | 5 | 5 | 45 |
| P6 | 5 | 15 | 55 |

El quantum del sistema es de 10 unidades. A misma prioridad el planificador es round robin.
a- Mostrar el orden de ejecución y las ráfagas de CPU de los procesos.

| Orden de ejecución | Ráfaga de CPU | Turnaround | Arribo | Tiempo espera |
|---|---|---|---|---|
| P1 | 15 | 15 | 0 | 0 |
| P2 | 5 | - | 0 | 15 |
| P3 | 10(10) | - | 20 | 0 |
| P4 | 10(10) | - | 25 | 5 |
| P3 | 5(5) | - | - | - |
| P5 | 5 | 5 | 45 | 0 |
| P4 | 5(5) | - | - | - |
| P6 | 15 | 15 | 55 | 0 |

| Orden de ejecución | Ráfaga de CPU | Turnaround | Arribo | Tiempo espera |
|---|---|---|---|---|
| P3 | 5 | 75 - 20 = 55 | - | - |
| P4 | 5 | 80 - 25 = 55 | - | - |
| P2 | 15 | 95 - 0 = 95 | - | - |

b.Turn around:
T $(15+5+15+55+55+95)/6 = 40$
c- Tiempo espera promedio
Tespera = $20/6 = 3.3$

# Ejercicio 4.

| Proceso | Ráfaga | Prioridad |
|---|---|---|
| P1 | 5 | 4 |
| P2 | 3 | 1 |
| P3 | 1 | 2 |
| P4 | 7 | 2 |
| P5 | 4 | 3 |

Los procesos arriban al sistema en el momento 0, en este orden: P1,P2,P3,P4,P5.
a)Mostrar el orden de ejecución y las ráfagas de CPU de los procesos si el planificador es FCFS, SJF, con prioridades no-apropiativo, round-robin (quatum = 2)

## FCFS(First come first served)

| Proceso | Ráfaga | Turn Around | Tiempo espera |
|---|---|---|---|
| P1 | 5 | 5 | 0 |
| P2 | 3 | 8 | 5 |
| P3 | 1 | 14 | 13 |
| P4 | 7 | 21 | 14 |
| P5 | 4 | 25 | 21 |

Turn around total

TurnArond = (5+8+14+21+25)/5 = 14.6

T espera = (5+13+14+21)/5 = 10.6

# SJB (Short Job First)

| Proceso | Ráfaga | Turn Around | Tiempo Espera |
|---|---|---|---|
| P3 | 1 | 1 | 0 |
| P2 | 3 | 4 | 3 |
| P5 | 4 | 7 | 4 |
| P1 | 5 | 12 | 7 |
| P4 | 7 | 19 | 12 |

Turn Around = (1+4+7+12+19)/5 = 8.6
T espera = (0+3+4+7+12)/5 = 5.2

## Prioridades no apropiativo.

| Proceso | Ráfaga | Turn Around | Tiempo Espera | Prioridad |
|---|---|---|---|---|
| P1 | 5 | 5 | 0 | 4 |
| P5 | 4 | 9 | 5 | 3 |
| P3 | 1 | 10 | 9 | 2 |
| P4 | 7 | 17 | 10 | 2 |
| P2 | 3 | 20 | 17 | 1 |

Turn Around (5+9+10+17+20)/ 5 = 12.2
Tiempo espera (0+5+9+10+17)/5 = 8.2

## Round Robin (Quantum = 2)

| Proceso | Ráfaga | Turn Around | TIempo Espera |
|---|---|---|---|
| P1 | 5-2 = 3 | - | 0 |
| P2 | 3-2=1 | - | 2 |
| P3 | 1 | 5 | 4 |
| P4 | 7-2=5 | - | 5 |
| P5 | 4-2=2 | - | 7 |
| P1 | 3-2=1 | - | - |
| P2 | 1 | 12 | - |
| P4 | 5-2=3 | - | - |
| P5 | 1 | 15 | - |
| P1 | 1 | 16 | - |
| P4 | 3-2=1 | - | - |

| Proceso | Ráfaga | Turn Around | TIempo Espera |
|---|---|---|---|
| P4 | 1 | 19 | - |

Turn Aroun (5+12+15+16+18)/5 = 13.2
Tiempo espera (0+2+4+5+7)/5 = 3.6

El algoritmo que produce el mínimo promedio del tiempo de espera es RR.

# Ejercicio 5.

## Averigue como puede ver el estado de cada proceso en ejecución de un sistema UNIX (Linux/MACOS), etc. ¿Qué estados posibles existen en UNIX? ¿Qué significa el estado zombie en UNIX según Tanenbaum?

Para listar todos lo procesos activos y su estado actual en un sistema UNIX, se puede utilizar los siguiente métodos:

```
ps -eo pid,user,stat,cmd //Listamos todos los procesos con ps
```

pid: ID del proceso
user: Usuario dueño del proceso
stat: Estado del proceso (R,S,D,Z)
cmd: comando ejecutado

LIstado por estado específico.

```
ps -eo pid,user,stat,cmd | grep ' R '  # Solo procesos en ejecución
(Running)
ps -eo pid,user,stat,cmd | grep ' S '  # Solo procesos durmiendo
(Sleeping)
ps -eo pid,user,stat,cmd | grep ' D '  # Solo procesos en E/S no
interrumpible
```

# Comando top.

```
top

htop //Mejorado con colores y navegación
```

EL comando top muestra un monitor interactivo de procesos.
Con la tecla P podemos ver los procesos ordenados por CPU.

Con la tecla M podemos ver los procesos ordenados por Memoria
con Q salimos.

## Procesos en jerarquía

```
pstree -p -a
```

Los estados de los procesos en UNIX son:

- R running. En ejecución o listo
- S sleeping. Esperando un evento (E/S)
- D Uninterruptible Sleep. Esperando (E/S) de disco, no se puede matar.
- Zombie. Proceso terminado pero no liberado por el padre.
- T Stopped: Detenido por una señal.
- I Idle. Proceso del kernel inactivo.

# Proceso zombie.

Un proceso **Zombie (Z)** es un proceso que **ya terminó su ejecución**, pero su entrada aún existe en la tabla de procesos del kernel porque su **proceso padre no ha leído su estado de salida** (usando `wait()` o similar).

## Características de un Zombie

✔ **No consume CPU (solo ocupa un PID en la tabla de procesos).**
✔ No se puede "matar" **con `kill -9` (ya está muerto, solo falta liberarlo).**
✔ Si hay muchos Zombies**, pueden agotar los PIDs disponibles (problema en sistemas antiguos).

Algunas causas de estas son:

- El padre no maneja correctamente SIGCHILD. PUede no usar wait o ignora la señal (signal(SIGCHIL,SIG_IGN))
- El padre está ocupado/bloqueado porque puede estar procesando otras tareas.
- El padre murió antes que el hijo. En este caoso, el hijo puede quedar huérfano y ser adoptado por "init" .
  Init es el primer proceso que se ejecuta en un sistema Linux/Unix, después del kernel. Entre sus responsabilidades, una de sus tareas es adoptar procesos huérfanos evitando proceso zombies.

```
ps - //Permite ver el proceso init
```

```c
#include <stdio.h>
#include <stdlib.h>
#include <signal.h>
#include <unistd.h>
#include<sys/types.h>
#include <sys/wait.h>

int verificarPalindromo(char arreglo[20], int longitud);
char arreglo [20];
int main (){
system("/bin/stty raw");

int i = 0, longitud, resultado = 1, pid1, pid2;
char letraI;


printf("Veamos si una palabra es o no capicua\n\r");
printf("Presione enter para terminar la palabra o número\n\r");


pid1 = fork();

if(pid1 == 0){ //EL proceso uno carga la palabra




while( i < 20){

printf("Ingrese la letra :)\n\r");

letraI = getchar();

if(letraI == 13){
break;
}else{



arreglo[i] = letraI;
i++;
}
}
printf("EL proceso uno duerme");
sleep(50);

}else {
waitpid(pid1,NULL,0);
```

```
if(i > 1){


longitud = i-1;

i =0;

printf("Primer caracter %c y ultimo %c
\n\r",arreglo[i],arreglo[longitud]);



pid2 = fork();

printf("Soy el proceso dos");

if (pid2 == 0){


//Verificamos si es palindromo

while(resultado && i != longitud){
printf("Primer caracter %c y ultimo %c\n\r",arreglo[i],arreglo[longitud]);

if (arreglo[i] != arreglo[longitud]){
resultado =0;
break;

}else{
i = i+1;
longitud--;
}

}

printf("EL proceso dos duerme");

sleep(80);

}else{
waitpid(pid2,NULL,0);
printf("SOy el padreee");
kill(pid1,SIGKILL);
kill(pid2,SIGKILL);

if (resultado ==1){
printf("Es palindromo la pabalra o capicua el numero :) \n\r");


}else{
```

```
printf("NO es palindromo ni capicua\n\r");

}
}
}else{
waitpid(pid1,NULL,0);
printf("Hay un solo caracter y es palindromo");
printf("SOy el padreee");

}
}

system("/bin/stty sane erase ^?");
}
```

```
#include <xinu.h>
void ingresarPalabra(void);
void verificarPalindromo(void);

char arreglo [20];
int resultado =1;
int longitud =0;


int  palindromoXinu(){
int pid1, pid2;
control(CONSOLE,TC_MODER, 0,0);

resume(create(ingresarPalabra,1024,20,"Proceso1",0));

receive();

resume(create(verificarPalindromo,1024,20,"Proceso2",0));


receive();


if (resultado ==1){
printf("Es palindromo la pabalra o capicua el numero :) \n\r");


}else{
printf("NO es palindromo ni capicua\n\r");

}

exit();
```

```
control(CONSOLE,TC_MODEC, 0,0);
}




void ingresarPalabra(void){
int i = 0, resultado = 1;
char letraI;


printf("Veamos si una palabra es o no capicua\n\r");


printf("Presione enter para terminar la palabra o número\n\r");


while( i < 20){

printf("Ingrese la letra :)\n\r");

letraI = getchar();

if(letraI == 13){
break;
}else{



arreglo[i] = letraI;
i++;
}
}

longitud = i-1;

}

void verificarPalindromo (void){
int i = 0;

while(resultado && i != longitud){
printf("Primer caracter %c y ultimo %c\n\r",arreglo[i],arreglo[longitud]);

if (arreglo[i] != arreglo[longitud]){
resultado =0;
break;

}else{
```

```
i = i+1;
longitud--;
}

}

}
```